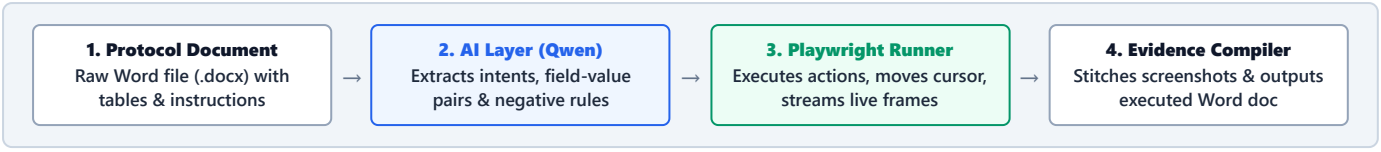
System Architecture: OnePV Validation Portal

How AI, Playwright, and Evidence Processing Work Together

1. ARCHITECTURE OVERVIEW & HIGH-LEVEL FLOW

The platform replaces manual protocol execution with a 3-layer architecture: **AI Interpretation**, **Browser Automation (Playwright)**, and **Evidence Assembly**. Instead of running hardcoded scripts with fixed values, the architecture dynamically bridges human-written Word protocol instructions to live web form interactions.



2. WHERE AI COMES INTO THE PICTURE

Validation protocols are written for human testers in descriptive English, not in code. The AI model (Qwen 2.5) functions as the **Protocol Interpreter and Dynamic Step Planner**:

A. Procedure & Intent Classification

The AI reads unstructured table rows from the uploaded Word document and identifies what each test is trying to do (e.g., login, general case details, reporter entry, patient demographics, product lookups, MedDRA coding, or questionnaire responses).

B. Dynamic Field-Value Extraction

Instead of hardcoding names, addresses, or medical terms in code, the AI extracts the exact values requested in the protocol (such as doctor names, patient ages, formulations, and terms like "Pyrexia") and pairs them with specific target UI fields.

C. Negative Test Rule Detection

The AI identifies whether a step is intended to pass via error (e.g., verifying that omitting mandatory dates blocks registration). It flags the expected validation rule so the runner knows an error message is the intended outcome, not a system failure.

D. Normalization into Action Contracts

Outputs a clean, structured JSON action plan with field IDs, control types (text input, select dropdown, search modal, toggle), and values. This plan is handed over directly to the browser runner without manual code generation.

3. WHERE PLAYWRIGHT TAKES ACTUAL ACTION

Playwright serves as the **Execution and Observability Engine**. It receives the structured plan from the AI layer and drives the live web application:

DOM Interaction & Controls: Playwright resolves DOM selectors on the live OnePV site, handles text typing, dropdown option matching, modal dialogs, and iframe searches (such as the MedDRA dictionary search and Product catalog selection).

Camera Motion & Viewport Centering

Before interacting with a form card, Playwright smoothly scrolls the browser viewport (`behavior: 'smooth', block: 'center'`) so reviewers can see the card in focus.

Virtual Mouse & Visual Pacing

Injects an in-DOM cursor that glides to target buttons and inputs, triggering a blue click-pulse ripple. Pauses 350ms between fields to maintain realistic, watchable pacing.

4. REAL-TIME STREAMING & LIVE DRAWER ARCHITECTURE

To let human reviewers monitor the headless execution in real time without opening remote desktop windows, the backend establishes a dedicated WebSocket stream:

WebSocket Frame Pipeline: After each significant field action, Playwright takes a lightweight 60% quality JPEG capture and broadcasts it via WebSocket to the React frontend. The Live View drawer displays this stream continuously, giving full visual feedback of the cursor moving, fields filling, and cards scrolling.

5. WHAT HAPPENS BEHIND THE SCENES: ERROR & DISCREPANCY HANDLING

The platform uses a decision matrix to differentiate between expected negative tests and unexpected application discrepancies:

Condition	Internal Mechanism	Resulting State
Expected Negative Test (e.g., Mandatory fields omitted)	The AI marks the step as expecting an error. Playwright checks for validation dialogs. When the expected alert appears, it verifies the text, takes evidence, dismisses the modal, and resets the form.	PASS Logged as verified validation behavior.
Unexpected Error Banner (e.g., 503 error, unhandled alert)	Playwright detects an unanticipated error popup or red alert banner. It takes an error screenshot and emits a `HUMAN_REVIEW_REQUIRED` event to the frontend.	PAUSED FOR REVIEW Prompts the user to review, acknowledge/continue, or halt.
Dropdown / Label Mismatches (e.g., "Dr" vs "Dr.")	Multi-tier resolution: Exact label match → Period variations → Exact value → Normalized substring lookup.	RESOLVED Selects correct option without breaking execution.
DOM & Network Latency	Replaces static sleeps with dynamic Playwright actionability checks (visible, attached, stable) before executing inputs.	STABLE Prevents timing race conditions during modal loads.

6. TWO-STAGE EVIDENCE CAPTURE & WORD REPORT RECONSTRUCTION

The final phase converts raw execution data into compliance-grade documentation:

Two-Stage Image Stitching

Playwright captures two separate states for critical steps: (1) the filled form before clicking Save/Submit, and (2) the confirmation popup showing the generated Case Number or alert. Both images are stitched vertically into one complete record.

Pristine Cursor Separation

The virtual cursor is visible in the live stream for the reviewer, but Playwright momentarily hides cursor DOM elements before taking screenshots, ensuring official document evidence remains clean.

Word Document Assembly (`python-docx`): Clones the uploaded protocol template and writes execution results directly into **Table 5 (Test Execution)**—filling Observed Results, Pass/Fail classifications, execution dates, tester initials, and embedding the stitched screenshot images. Any flagged discrepancies are written into **Table 8 (Discrepancy Log)**.